- **函数的基本语法**
  - 函数的声明语法

```
function 函数名() {
    函数体
}
```

    - 例

```
function sayHi() {
    document.write('hai~~')
}
```

  - 函数名命名规范
    - 和变量命名基本一致
    - 尽量小驼峰式命名法
    - 前缀应该为动词
    - 命名建议：常用动词约定

动词	含义
can	判断是否可执行某个动作
has	判断是否含义某个值
is	判断是否为某个值
get	获取某个值
set	设置某个值
load	加载某些数据

```
function getName() {}
function addSquares() {}
```

- 函数的调用语法

```
// 函数调用，这些函数体内的代码逻辑会被执行
函数名()
```

  - 注意：声明（定义）的函数必须调用才会真正被执行，使用 () 调用函数
  - 例

```
sayHi()
// 可以重复被调用，多少次都可以
sayHi();
```

- 函数体
  - 函数体是函数的构成部分，它负责将相同或相似代码“包裹”起来，直到函数调用时函数体内的代码才会被执行。函数的功能代码都要写在函数体当中。

```
// 打招呼
function sayHi() {
  console.log('嗨~');
}
sayHi()
```

函数体

- **函数封装思想**
  - 目的：通过这段代码封装成函数，体会函数的好处

```
<style>
    .left {
        width: 30px;
        height: 30px;
        text-align: center;
        line-height: 30px;
        background-color: pink;
    }

    .right {
        width: 30px;
        height: 30px;
        text-align: center;
        line-height: 30px;
        background-color: purple;
    }
</style>
```

抽取-封装

```
.btn {
    width: 30px;
    height: 30px;
    text-align: center;
    line-height: 30px;
}
.left {
    background-color: pink;
}
.right {
    background-color: purple;
}
```

- **有参数的函数声明和调用**
  - 声明语法

```
function 函数名(参数列表) {
    函数体
}
```

    - 参数列表
      - 传入数据列表
      - 声明这个函数需要传入几个数据
      - 多个数据用逗号隔开
    - 例
      - 单个参数

```
function getSquare(num1) {
    document.write(num1 * num1)
}
```

- 多个参数

```
function getSum(num1, num2) {
    document.write(num1 + num2)
}
```

- 调用语法

```
函数名(传递的参数列表)
```

  - 例

```
getSquare(8)
```

```
getSum(10, 20)
```

```
function getSum(num1, num2) {
    document.write(num1 + num2)
}

getSum(10, 20)
```

  - 调用函数时，需要传入几个数据就写几个，用逗号隔开

- 执行过程

```
function getSum(num1, num2) {
    document.write(num1 + num2)
}

getSum(10, 20)
```

```
// 类似执行了如下代码
num1 = 10
num2 = 20
```

- **形参和实参**

- 形参：声明函数时写在函数名右边小括号里的叫形参（形式上的参数）
- 实参：调用函数时写在函数名右边小括号里的叫实参（实际上的参数）
- 形参可以理解为是在这个函数内声明的变量（比如 num1 = 10）
- 实参可以理解为是给这个变量赋值
- 开发中尽量保持形参和实参个数一致

- **用return返回数据**
  - 当函数需要返回数据出去时，用return关键字
  - 语法

```
return 数据
```

```
return 20
```

  - 怎么使用呢?

```
function getSum(x, y) {
    return x + y
}
let num = getSum(10, 30)
document.write(num)
```

  - 细节：
    - 在函数体中使用 return 关键字能将内部的执行结果交给函数外部使用
    - 函数内部只能出现 1 次 return，并且 return 后面代码不会再被执行，所以 return 后面的数据不要换行写
    - return会立即结束当前函数
    - 函数可以没有 return，这种情况默认返回值为 undefined

- **内置函数概念**
  - 什么是内置函数
    - JavaScript内部封装好的一些函数，让开发者直接调用就完成对应功能
    - 例：prompt() alert() document.write() Number() Boolean()
    - 内置函数带括号的，意味着可以传递参数
  - 接下来要学的新内置函数
    - parseInt

        - parseFloat

- **内置函数之数字提取**
    - parseInt()
        - 是一个返回值为number类型的函数
        - 作用：从字符串中提取出整数
        - 细节：
            - 字符串内容从左到右依次分析是否数字，直到遇到非数字停止
            - 如果字符串不是以数字开头，得到NaN
    - parseFloat()
        - 是一个返回值为number类型的函数
        - 作用：从字符串中提取数字（包括小数）
        - 细节：
            - 字符串内容从左到右依次分析是否数字，直到遇到第二个小数点或者非数字停止
            - 如果字符串不是以数字开头，得到NaN